JENKINS — COMPLETE STUDY NOTES

DevOps Training | Cognizant | Allen

PHASE 1 — THE MENTAL MODEL

What is Jenkins and Why Does It Exist?

Jenkins is an open-source CI/CD automation server written in Java. It watches your source code repository and automatically triggers a pipeline of build, test, quality analysis, packaging, and deployment steps every time code is pushed. It eliminates manual integration work, gives teams immediate feedback on code quality, and plugs into virtually any tool in the DevOps ecosystem via its plugin architecture.

Three Reasons Jenkins Dominates

- **Open source and free** — No licensing cost. Any team, any size.
- **Plugin ecosystem** — 1800+ plugins — Git, Maven, Docker, Kubernetes, Azure, Slack, Nexus, and more.
- **Language and platform agnostic** — Works with Java, .NET/C#, Python, Node.js, Ruby, and any stack.

Build Maturity Ladder

- Rung 1 — No Build Process: IDE manual deployment, "works on my machine" problem
- Rung 2 — Minimal Build Process: Maven/Gradle/NPM used, but a "build guy" runs it manually
- Rung 3 — Continuous Inspection: Jenkins auto-detects push, builds, runs tests and static analysis
- Rung 4 — Continuous Integration: Inspection + branching strategy + release management policy
- Rung 5 — Continuous Delivery: Auto-packaged and ready; human presses the deploy button
- Rung 6 — Continuous Deployment: Fully automated to production, no human approval

⚠ NOTE: Continuous Inspection and Continuous Integration are NOT the same. CI = Inspection + branching strategy + release policy. Interviewers test this distinction regularly.

End-to-End Automated Pipeline

```
Developer pushes code to Azure Repos
         ↓
Jenkins detects the change (Poll SCM or Webhook)
         ↓
Jenkins pulls the latest source code
         ↓
Build Tool (Maven) compiles the code
         ↓
Unit Tests run automatically (JUnit)
         ↓
Static Code Analysis runs (CheckStyle, PMD, FindBugs)
         ↓
Artifact is packaged (.WAR / .JAR file)
         ↓
Artifact is published to a repository (Nexus / Artifactory)
         ↓
Artifact is deployed to application server (Tomcat / Azure App Service)
         ↓
Team gets a notification (email / dashboard status)
```

⚠ NOTE: In your Azure/Cognizant environment: Azure Repos → Jenkins → Maven → Azure Artifacts → Azure App Service / Azure VM. The Azure DevOps plugin connects Jenkins to Azure Repos as the SCM source.

PHASE 2 — CORE BUILDING BLOCKS

Building Block 1 — Jobs (Projects)

A Job is the central unit of work in Jenkins. Everything Jenkins does is defined inside a Job. Jenkins uses "Job" and "Project" interchangeably. The Job is the definition (the recipe card). The execution of that definition is called a Build (one cooking session).

Job Types Comparison Table

Job Type	When to use it
Freestyle Project	Simple tasks, GUI-configured, no code required
Maven Project	Java Maven builds — Jenkins understands Maven lifecycle
Pipeline Project	Code-defined pipeline (Jenkinsfile) — Jenkins 2.0 standard
Multibranch Pipeline	One pipeline per branch, auto-discovered from repo
Multiconfiguration	Same job across multiple OS/JDK configurations
Folder	Organises jobs into groups like a directory
GitHub Organization	Scans entire GitHub org, creates Multibranch per repo

⚠ **NOTE: Freestyle is what beginners learn and older setups use. Pipeline (Jenkinsfile) is the modern Jenkins 2.0 standard. Know both — interviewers ask about both.**

Six Sections of a Freestyle Job

```
Section                    What it controls
──────────────────────────────────────────────────────────
General                    Job name, description, parameters, build retention
Source Code Management     Repository URL, branch, credentials
Build Triggers             What event causes Jenkins to run this job
Build Environment          Environment setup before the build runs
Build Steps                The actual commands to execute
Post-build Actions         What to do after the build finishes
```

Building Block 2 — Builds

Build States

```
State          Colour       Meaning
──────────────────────────────────────────────────────────
Success        Blue/Green   All steps completed without errors
Unstable       Yellow       Build completed but tests failed or thresholds breached
Failure        Red          Build broke — compilation error, critical failure
Aborted        Grey         Manually stopped before finishing
```

⚠ **NOTE: Failure = build could not complete (compilation error). Unstable = build completed but results unacceptable (tests failed). This distinction is a very common interview question.**

What Jenkins Stores Per Build

- **Console Output** — Full log of every command — primary debugging tool
- **Test Results** — JUnit XML parsed as trend graph
- **Archived Artifacts** — .WAR/.JAR files produced by the build
- **Change Set** — Which commits triggered this build and what files changed
- **Build Timestamp** — Start time and duration

Building Block 3 — Jenkins Architecture (Master + Agents)

Jenkins is a distributed system. The Master (Controller) is the brain — stores configs, schedules builds, serves the UI. Agents (Nodes) are the workers — they receive instructions and execute builds. The Master does not run builds in production setups.

```
                    ┌──────────────────┐
                    │   JENKINS        │
                    │   MASTER         │
                    │  (Controller)    │
                    │ • Web UI         │
                    │ • Job configs    │
                    │ • Scheduling     │
                    └──────────────────┘
                              │
                ┌─────────────┼─────────────┐
                ▼             ▼             ▼
        ┌──────────┐   ┌──────────┐   ┌──────────┐
        │ AGENT 1  │   │ AGENT 2  │   │ AGENT 3  │
        │(Linux VM)│   │(Win VM)  │   │(Docker)  │
        └──────────┘   └──────────┘   └──────────┘
```

⚠ NOTE: Modern term is Controller/Agent. The word "Slave" is deprecated. Always use Controller/Agent in interviews. If you see Master/Slave in docs, it means the same thing.

Azure Architecture

- **Jenkins Master** — Azure VM (Ubuntu/Windows Server) or Azure Container Instance
- **Jenkins Agents** — Azure VM Scale Set, Azure Container (Docker), or AKS Pod

Building Block 4 — Plugins

Out of the box, Jenkins is minimal. Everything — Git, Maven, Docker, email, test publishing, Azure — comes from Plugins. Think of Jenkins as a smartphone OS and Plugins as apps. 1800+ plugins available.

Plugin Manager Tabs

```
Tab           What it shows
──────────────────────────────────────────────────
Available     All installable plugins not yet installed
Installed     All plugins currently on this Jenkins instance
Updates       Installed plugins with newer versions available
Advanced      Manual plugin upload (.hpi), proxy settings
```

⚠ NOTE: Plugins are the most common source of Jenkins instability. Outdated or incompatible plugins are the most frequent cause of unexpected Jenkins behaviour. Always restart Jenkins after installing plugins.

Essential Plugins

```
Plugin                        What it enables
```

```
──────────────────────────────────────────────────────────
Git Plugin                      Connect to Git / GitHub / Azure Repos
Maven Integration Plugin        Maven project type support
Pipeline Plugin                 Pipeline project type + Jenkinsfile
Multibranch Pipeline Plugin     Per-branch pipeline from repo scan
Blue Ocean Plugin               Modern visual pipeline UI
Deploy to Container Plugin      Deploy WAR to Tomcat
JUnit Plugin                    Test result parsing and trend graph
CheckStyle Plugin               Static analysis results in UI
Cobertura Plugin                Code coverage reports
Email Extension Plugin          Rich email notifications
Azure DevOps Plugin             Azure Repos SCM + Azure Artifacts
Timestamper Plugin              Timestamps in Console Output
Workspace Cleanup Plugin        cleanWs() step in pipelines
```

PHASE 3 — SYNTAX AND CONFIGURATION

Freestyle Job — General Section

```
field: Project Name         → URL-safe, hyphens not spaces
field: Description          → Human-readable note shown on job page
field: Discard Old Builds   → Log Rotation strategy
  Days to keep builds: 30
  Max # of builds to keep: 10
field: Parameterized Build → String / Boolean / Choice / Password / File
  Name: DEPLOY_ENV
  Default Value: staging
```

⚠ NOTE: Build retention is not optional in production. Without Discard Old Builds, Jenkins will eventually fill the disk and crash.

Freestyle Job — SCM Section

```
SCM: Git
Repository URL: https://dev.azure.com/ORG/PROJECT/_git/REPO
Credentials: azure-repos-token
Branch Specifier: */main
  */main    = main branch
  */dev     = dev branch
  **        = all branches (Multibranch)
```

⚠ NOTE: Never hardcode passwords in the Repository URL (https://user:pass@...). This exposes credentials in Console Output. Always use the Credentials field.

Freestyle Job — Build Triggers Section

```
Build periodically:   Runs on schedule regardless of code changes
Poll SCM:             Checks repo first — only builds if new commits found
GitHub hook trigger:  Webhook from repo — immediate, no polling delay
Build after other:    Creates upstream/downstream job chain
```

```
Cron syntax format:   MINUTE  HOUR  DAY-OF-MONTH  MONTH  DAY-OF-WEEK
H * * * *    →  Once per hour
H/5 * * * *  →  Every 5 minutes
H 8 * * 1-5  →  Every weekday at approximately 8 AM
H/15 * * * * →  Every 15 minutes
```

⚠ NOTE: Poll SCM only builds if code changed. Build Periodically always builds on schedule even if nothing changed. This distinction is a frequent interview question.

Freestyle Job — Build Steps + Jenkins Built-in Variables

```
# Invoke Maven
Maven Version: Maven-3.8.6
Goals: clean package
POM: pom.xml

# Shell script step
echo "Build: ${BUILD_NUMBER}"
echo "Job: ${JOB_NAME}"
echo "Branch: ${GIT_BRANCH}"
echo "Workspace: ${WORKSPACE}"
```

Jenkins Built-in Environment Variables

```
BUILD_NUMBER   →  Sequential build number (1, 2, 3...)
BUILD_URL      →  Full URL to this build page in Jenkins
JOB_NAME       →  Name of the job being built
WORKSPACE      →  Absolute path to workspace on Agent
GIT_COMMIT     →  Full SHA of the commit being built
GIT_BRANCH     →  Branch that was checked out
NODE_NAME      →  Name of the Agent running this build
BRANCH_NAME    →  Branch name (Multibranch pipelines ONLY)
```

Freestyle Job — Post-build Actions

```
# Publish JUnit test results
Test report XMLs: **/target/surefire-reports/*.xml

# Archive artifacts
Files to archive: target/*.war
Fingerprint all archived artifacts: checked

# Email notification
Recipients: allen@cognizant.com
Send email for every unstable build: checked
Send separate emails to individuals who broke the build: checked

# Trigger downstream job
Projects to build: my-java-app-deploy
Trigger only if build is stable: checked
```

Declarative Pipeline — Complete Jenkinsfile

```
pipeline {
    agent any

    environment {
        APP_NAME   = "my-java-app"
        DEPLOY_ENV = "staging"
        MAVEN_OPTS = "-Xmx512m"
    }

    options {
        timeout(time: 30, unit: "MINUTES")
        retry(2)
        skipDefaultCheckout(false)
        disableConcurrentBuilds()
    }

    stages {
        stage("Checkout") {
            steps {
                echo "Branch: ${env.BRANCH_NAME}"
                checkout scm
            }
        }
        stage("Build") {
            steps { sh "mvn clean compile -B" }
        }
        stage("Test") {
            steps { sh "mvn test -B" }
            post {
                always {
                    junit testResults: "**/target/surefire-reports/*.xml",
                          allowEmptyResults: true
                }
            }
        }
        stage("Code Quality") {
            parallel {
                stage("CheckStyle") { steps { sh "mvn checkstyle:check -B" } }
                stage("PMD")        { steps { sh "mvn pmd:check -B" } }
            }
        }
        stage("Package") {
            steps { sh "mvn package -DskipTests -B" }
            post {
                success {
                    archiveArtifacts artifacts: "target/*.war",
                                     fingerprint: true,
                                     onlyIfSuccessful: true
                }
            }
        }
        stage("Deploy to Staging") {
            when { anyOf { branch "main"; branch "development" } }
```

```
            steps { sh "curl -u user:pass -T target/app.war URL/deploy?path=/app"
}
        }
        stage("Deploy to Production") {
            when { branch "main" }
            steps {
                timeout(time: 10, unit: "MINUTES") {
                    input message: "Approve production deployment?",
                          ok: "Deploy to Production",
                          submitter: "admin,allen"
                }
                sh "curl -u user:pass -T target/app.war PROD-URL/deploy?path=/app"
            }
        }
    }

    post {
        always   { cleanWs() }
        success  { mail to: "allen@cognizant.com", subject: "SUCCESS:
${env.JOB_NAME}", body: "${env.BUILD_URL}" }
        failure  { mail to: "allen@cognizant.com", subject: "FAILURE:
${env.JOB_NAME}", body: "${env.BUILD_URL}" }
        unstable { mail to: "allen@cognizant.com", subject: "UNSTABLE:
${env.JOB_NAME}", body: "${env.BUILD_URL}" }
    }
}
```

Declarative Pipeline — Block Reference

`pipeline { }` → Outermost mandatory wrapper — everything lives inside

`agent any` → Run on any available Agent/Node

`agent { label "x" }` → Run only on Agents tagged with this label

`agent { docker { } }` → Spin up Docker container, run steps inside it, destroy after

`agent none` → No pipeline-level agent — each stage declares its own

`environment { }` → Pipeline-level variables available in ALL stages

`options { }` → Pipeline behaviour settings applied before any stage

`timeout(time,unit)` → Abort entire pipeline if it runs longer than this

`retry(n)` → Auto-retry pipeline up to n times on failure

`disableConcurrentBuilds()` → Prevent two builds running simultaneously on same branch

`stages { }` → Container for all stage blocks — must have at least one

`stage("name") { }` → Named phase shown in Jenkins UI and Blue Ocean

`steps { }` → The actual commands to run inside a stage

`sh "..."` → Run shell command on Linux Agent

`bat "..."` → Run batch command on Windows Agent

`checkout scm` → Check out the triggering branch — only works in Multibranch

`when { }` → Conditional — stage only runs if condition is true

`branch "main"` → when condition: only run on the main branch

`allOf { }` → Logical AND — all conditions must be true

`anyOf { }` → Logical OR — at least one condition must be true

`parallel { }` → Run nested stages simultaneously — reduces total build time

`input message,ok` → Pause and wait for human approval before continuing

`submitter: "a,b"` → Only these Jenkins usernames can approve the input gate

`post { }` → After-build actions — at pipeline or stage level

`always { }` → Runs after every build regardless of result

`success { }` → Runs only when result is SUCCESS

`failure { }` → Runs only when result is FAILURE

`unstable { }` → Runs only when result is UNSTABLE (tests failed)

`changed { }` → Runs when result differs from previous build

`fixed { }` → Runs when current is SUCCESS but previous was FAILURE

`cleanWs()` → Delete workspace directory on Agent after build

⚠ NOTE: In Groovy: double-quoted strings expand variables "${env.VAR}". Single-quoted strings do not. Always use double quotes when you need variable interpolation in Groovy expressions.

Scripted vs Declarative Pipeline Comparison

	Declarative	Scripted
Syntax wrapper	pipeline { }	node { }
Structure	Strict and opinionated	Freeform Groovy code
Error messages	Clearer	Harder to debug
when conditions	Built-in when { }	Manual if/else in Groovy
Post conditions	Built-in post { }	Try/catch/finally blocks
Learning curve	Lower	Higher
Recommended for	Everyone — modern std	Advanced users needing full Groovy
Jenkins version	Jenkins 2.0+	All versions

Multibranch Pipeline

A Multibranch Pipeline scans an entire repository, finds every branch containing a Jenkinsfile, and automatically creates a separate child pipeline job for each. When a branch is deleted, its job is removed automatically.

```
Repository
├── main          → Jenkins creates: my-app/main
├── development   → Jenkins creates: my-app/development
├── feature/login → Jenkins creates: my-app/feature-login

Configuration:
Branch Sources → Git → Repository URL + Credentials
Discover branches: All branches
Filter by name: (main|development|feature/.*)
Script Path: Jenkinsfile
Scan interval: 1 minute
Orphaned Item Strategy: 7 days, 5 max
```

Branch Behaviour Matrix

Branch	Build	Test	Deploy Staging	Deploy Production
feature/*	YES	YES	NO	NO
development	YES	YES	YES	NO
main	YES	YES	YES	YES (with approval)

⚠ NOTE: env.BRANCH_NAME is only automatically populated in Multibranch Pipeline jobs. In a regular single Pipeline job it is not set unless configured manually.

Blue Ocean UI

- Visual pipeline UI plugin installed separately
- Both classic UI and Blue Ocean exist simultaneously
- Stages shown left to right as connected colour-coded boxes
- Click any stage box to see ONLY that stage's console output
- Parallel stages shown as stacked rows within same column
- Input approval gate shown as pause icon on stage box
- Branch/PR tabs for Multibranch pipeline jobs

⚠ NOTE: The per-stage log in Blue Ocean is the biggest practical advantage over classic Jenkins. Classic shows one wall of text for the entire build. Blue Ocean lets you click the red stage and see only that stage's output.

Jenkinsfile Validation — Pipeline Linter

```
# curl validation command
curl -X POST \
     -u admin:YOUR-API-TOKEN \
     http://localhost:8080/pipeline-model-converter/validate \
     --data-urlencode "jenkinsfile=$(cat Jenkinsfile)"

# Success response:
Jenkinsfile successfully validated.

# Error response example:
WorkflowScript: 45: unexpected token: when @ line 45, column 9.
```

VS Code Jenkins Pipeline Linter Connector — settings.json

```
{
    "jenkins.pipeline.linter.connector.url":
        "http://localhost:8080/pipeline-model-converter/validate",
    "jenkins.pipeline.linter.connector.user": "admin",
    "jenkins.pipeline.linter.connector.pass": "your-api-token",
    "jenkins.pipeline.linter.connector.crumbUrl":

"http://localhost:8080/crumbIssuer/api/json/xpath=concat(//crumbRequestField,\":\"
,//crumb)"
}
```

⚠ NOTE: The when block must always be a direct child of a stage block. It cannot be placed inside a parallel block. This is the most common Jenkinsfile structural error.

PHASE 4 — WORKFLOW

Workflow A — Jenkins Setup from Scratch

Step 1 — Verify Prerequisites

```
java -version    # Java 11 minimum, 17 recommended
git --version
mvn -version     # If "command not found" — M2_HOME/PATH not set
```

Step 2 — Getting Started Wizard

```
Navigate to: http://localhost:8080

Initial admin password (Linux):   sudo cat
/var/lib/jenkins/secrets/initialAdminPassword
Initial admin password (Windows): C:\Program
Files\Jenkins\secrets\initialAdminPassword
```

```
Screen 1: Unlock Jenkins (paste initial password)
Screen 2: Install suggested plugins
Screen 3: Create admin account (use real email)
Screen 4: Instance Configuration — set Jenkins URL to public IP if Azure VM
```

Step 3 — Global Tool Configuration

```
Manage Jenkins → Global Tool Configuration

JDK:   Name=JDK-11    JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
Maven: Name=Maven-3.8.6  MAVEN_HOME=/opt/maven
Git:   Name=Default   Path to Git executable=git

OR tick "Install automatically" for cloud agents (Azure VM Scale Set)
```

Step 4 — Store Credentials

```
Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Kind: Username with password
Username: allen
Password: [Azure DevOps Personal Access Token]
ID: azure-repos-token   ← always set manually
Description: Azure Repos PAT for pipeline SCM checkout
```

⚠ NOTE: Jenkins encrypts credentials at rest. They are never shown in plain text after saving and are masked as **** in Console Output. To update, replace — you cannot retrieve the original value.**

Step 5 — SMTP Email Configuration

```
Manage Jenkins → Configure System → E-mail Notification

SMTP server: smtp.office365.com  (Microsoft 365 / Outlook)
SMTP port: 587 (TLS) or 465 (SSL)
Use SMTP Authentication: checked
Username: allen@cognizant.com
Password: [App password — NOT your login password]

→ Always click "Test configuration by sending test e-mail"
```

Workflow B — Freestyle Job End to End

Step-by-Step Sequence

```
Step 1:  Dashboard → New Item → name: my-java-app → Freestyle project → OK
Step 2:  General: description, Discard Old Builds (30 days, 10 max), parameters
Step 3:  SCM: Git, Azure Repos URL, azure-repos-token credential, */main branch
Step 4:  Build Triggers: Poll SCM H/5 * * * * or GitHub hook trigger
Step 5:  Build Environment: Delete workspace before build + Add timestamps
Step 6:  Build Steps: Invoke Maven (clean package) + Execute shell (diagnostic
echo)
Step 7:  Post-build: JUnit **/target/surefire-reports/*.xml
                     Archive target/*.war (fingerprint)
```

```
                      Email allen@cognizant.com on unstable/failure
                      Trigger downstream: my-java-app-deploy (stable only)
Step 8:  Save
Step 9:  Build with Parameters → DEPLOY_ENV=staging → Build
Step 10: Monitor Console Output → git clone → Maven → BUILD SUCCESS
Step 11: Build page → Artifacts (WAR download) + Test Result + Changes
Step 12: Job page → Test Result Trend graph (spot regressions across builds)
```

⚠ NOTE: After entering Repository URL + Credentials, Jenkins immediately validates the connection. If you see a red error under the URL field, fix it before saving.

Workflow C — Pipeline Job End to End

Step 1 — Write and Validate Jenkinsfile Locally

```
# Place Jenkinsfile in repository root
ls -la Jenkinsfile

# Validate before pushing
curl -X POST -u admin:API-TOKEN \
     http://localhost:8080/pipeline-model-converter/validate \
     --data-urlencode "jenkinsfile=$(cat Jenkinsfile)"
```

Step 2 — Push to Repository

```
git add Jenkinsfile
git commit -m "ci: add Jenkins declarative pipeline"
git push origin main

# Create development branch
git checkout -b development
git push origin development
```

Step 3 — Create Multibranch Pipeline Job

```
New Item → my-java-app → Multibranch Pipeline → OK

Branch Sources → Add source → Git
  Repository URL: https://dev.azure.com/ORG/PROJECT/_git/REPO
  Credentials: azure-repos-token
  Discover branches: All branches
  Filter by name regex: (main|development|feature/.*)

Build Configuration:
  Mode: by Jenkinsfile
  Script Path: Jenkinsfile

Scan Triggers: Periodically 1 minute
Orphaned Item Strategy: 7 days, 5 max

Save → Jenkins immediately scans repo → creates child jobs per branch
```

Step 4 — Monitor in Blue Ocean

```
Dashboard → Open Blue Ocean → my-java-app → main branch → build #1

Stages shown left to right:
Checkout → Build → Test → Code Quality → Package → Deploy Staging → Deploy
Production

Code Quality shows parallel:
   ┌──────────────┐
   │  CheckStyle  │  ← parallel branch 1
   ├──────────────┤
   │     PMD      │  ← parallel branch 2
   └──────────────┘


Deploy Production: ⏸ Paused — waiting for input
  Click "Deploy to Production" to proceed
  Click "Abort" to skip deployment
```

⚠ NOTE: The input step holds an Agent executor while waiting for approval. Always wrap it with timeout() to prevent it waiting indefinitely and blocking other builds.

PHASE 5 — REAL WORLD STRUCTURE

Folder Structure and Job Organisation

```
Jenkins Dashboard
│
├── 🗀 platform-team
│     ├── 🗀 infrastructure
│     │     ├── terraform-azure-networking
│     │     └── terraform-azure-aks
│     └── 🗀 monitoring
│           └── prometheus-deploy
│
├── 🗀 product-team-alpha
│     ├── 🗀 backend
│     │     └── 🔀 alpha-backend-api  ← Multibranch Pipeline
│     ├── 🗀 frontend
│     │     └── 🔀 alpha-frontend-app
│     └── 🗀 release
│           ├── alpha-release-staging
│           └── alpha-release-production
│
└── 🗀 shared-utilities
      ├── database-backup
      └── jenkins-backup
```

⚠ **NOTE: When referencing a job inside a folder from another job, use the full path with forward slashes: product-team-alpha/backend/alpha-backend-api. Using just the job name will fail.**

Views

```
View type       Purpose
──────────────────────────────────────────────────────────
All             Default — shows everything at this level
List View       Filtered subset — manual tick or regex filter
My View         Only jobs the current logged-in user can run

Regex filter example: alpha-.*
Matches all jobs starting with alpha- automatically
Weather column: sun/cloud/storm — recent build health indicator
```

Shared Libraries

Shared Libraries extract common pipeline logic into a separate Git repository. Teams import the library into their Jenkinsfiles with @Library and call its functions. Update once — all pipelines benefit.

Library Repository Structure

```
jenkins-shared-library/
├── vars/                    ← callable functions (filename = function name)
│     ├── buildMavenProject.groovy
│     ├── deployToTomcat.groovy
│     ├── sendBuildNotification.groovy
│     └── runCodeQuality.groovy
├── src/                     ← Groovy classes for complex logic
│     └── org/cognizant/utils/
│           └── AzureUtils.groovy
└── resources/               ← static files (HTML templates etc.)
      └── email-templates/
```

⚠ **NOTE: The directory names vars/, src/, and resources/ are fixed. Jenkins specifically looks for these names. You cannot rename them.**

vars/ Function Pattern

```
// vars/buildMavenProject.groovy
def call(Map config = [:]) {
    def mavenVersion = config.mavenVersion ?: "Maven-3.8.6"
    def goals        = config.goals        ?: "clean package"
    def pomFile      = config.pomFile      ?: "pom.xml"
    withMaven(maven: mavenVersion) {
        sh "mvn ${goals} -f ${pomFile} -B"
```

```
    }
}
```

`def call(Map config = [:])` → Mandatory method name. Jenkins calls this when Jenkinsfile uses the function.

`config.key ?: "default"` → Elvis operator — use caller's value if provided, otherwise use default.

`error("msg")` → Immediately fail the pipeline if a required parameter was not provided.

withCredentials() in Library Functions

```
withCredentials([
    usernamePassword(
        credentialsId: credId,
        usernameVariable: "TOMCAT_USER",
        passwordVariable: "TOMCAT_PASS"
    )
]) {
    sh "curl -u ${TOMCAT_USER}:${TOMCAT_PASS} -T ${warFile} ${url}"
}
```

⚠ NOTE: Never put credentials directly in a Jenkinsfile as plain text strings. Git history is permanent — a password committed to a repo can be retrieved from history even after deletion. Always use withCredentials().

Registering Library in Jenkins

```
Manage Jenkins → Configure System → Global Pipeline Libraries → Add

Name:            cognizant-shared-lib
Default version: main          ← pin to release tag (v1.2.0) in production
Load implicitly: unchecked     ← always prefer explicit import
Retrieval:       Modern SCM → Git
Repository URL:  https://dev.azure.com/ORG/devops/_git/jenkins-shared-library
Credentials:     azure-repos-token
```

Using Library in a Jenkinsfile

```
@Library("cognizant-shared-lib@main") _    ← must be FIRST line, trailing _
required

pipeline {
    agent any
    stages {
        stage("Build") {
            steps {
```

```
                buildMavenProject(mavenVersion: "Maven-3.8.6", goals: "clean
package")
            }
        }
        stage("Deploy") {
            when { branch "main" }
            steps {
                deployToTomcat(
                    warFile:   "target/my-app.war",
                    tomcatUrl: "http://20.10.45.123:8080",
                    appPath:   "/my-app",
                    credId:    "tomcat-manager-credentials"
                )
            }
        }
    }
    post {
        always { sendBuildNotification(recipients: "team@cognizant.com");
cleanWs() }
    }
}
```

Before vs After Shared Library

```
WITHOUT Shared Library          WITH Shared Library
──────────────────────────────────────────────────────
Each Jenkinsfile: ~80 lines     Each Jenkinsfile: ~35 lines
Tomcat URL in every file        Tomcat URL in library once
Email template duplicated x15   Email template in one place
Update: edit 15 files, 15 PRs  Update: edit 1 file, 1 PR
```

REFERENCE PATTERNS

Variable Referencing

Context	Format	Example
Bash shell	${VARIABLE_NAME}	${BUILD_NUMBER}
Windows batch	%VARIABLE_NAME%	%BUILD_NUMBER%
Groovy/Jenkinsfile	env.VARIABLE_NAME	env.BUILD_NUMBER
Pipeline parameters	params.PARAM_NAME	params.DEPLOY_ENV
Groovy interpolation	"${env.VARIABLE_NAME}"	"${env.JOB_NAME}"

⚠ NOTE: Single-quoted strings in Groovy do NOT expand variables. Double-quoted strings DO. sh 'echo $BUILD_NUMBER' works (shell expands it). echo '${env.JOB_NAME}' in Groovy prints literal text.

Credential Referencing

```
Step                Format
──────────────────────────────────────────────────────────────────
git/checkout step  credentialsId: "id-string"

withCredentials([
    usernamePassword(
        credentialsId: "my-cred-id",
        usernameVariable: "MY_USER",
        passwordVariable: "MY_PASS"
    )
]) {
    sh "curl -u ${MY_USER}:${MY_PASS} ..."
}
```

Maven Goal Reference

```
Goal                  What it does
──────────────────────────────────────────────────────────────────
mvn clean             Delete target/ directory (remove previous build
output)
mvn compile           Compile source code only
mvn test              Compile + run unit tests
mvn package           compile + test + package into .WAR or .JAR
mvn install           package + install to local .m2 repository
mvn deploy            install + push to remote Nexus/Artifactory
mvn clean package     Most common CI goal — clean then full package
mvn package -DskipTests  Package without re-running tests
mvn checkstyle:check Run CheckStyle static analysis
mvn pmd:check         Run PMD static analysis
mvn -B                Batch mode — suppress download progress output
mvn -f path/pom.xml   Use specific POM file path
```

Cron Syntax Quick Reference

```
Format:  MINUTE  HOUR  DAY-OF-MONTH  MONTH  DAY-OF-WEEK

H * * * *          →  Once per hour
H/5 * * * *        →  Every 5 minutes
H/15 * * * *       →  Every 15 minutes
H 8 * * 1-5        →  Every weekday at approximately 8 AM
H 6 * * 0          →  Every Sunday at approximately 6 AM
0 0 * * *          →  Every day at midnight exactly

H = hash (Jenkins picks consistent random value within window)
/ = step     - = range     , = list     * = every value
```

ALL CRITICAL NOTES — INTERVIEW WATCH LIST

⚠ **NOTE:** Continuous Inspection ≠ Continuous Integration. CI = Inspection + branching strategy + release policy.

⚠ **NOTE:** Failure = build could not complete (compilation error). Unstable = build completed but results unacceptable (tests failed).

⚠ **NOTE:** Modern Jenkins terminology: Controller (not Master), Agent (not Slave). Always use Controller/Agent in interviews.

⚠ **NOTE:** Plugins are the most common source of Jenkins instability. Always restart Jenkins after installing plugins.

⚠ **NOTE:** Never hardcode passwords in the Repository URL. Always use the Jenkins Credentials field.

⚠ **NOTE:** Build retention (Discard Old Builds) is mandatory in production — without it Jenkins fills the disk.

⚠ **NOTE:** Poll SCM only builds if code changed. Build Periodically always builds on schedule even if nothing changed.

⚠ **NOTE:** The when block must be a direct child of a stage block — not inside parallel, not inside steps.

⚠ **NOTE:** env.BRANCH_NAME is only populated in Multibranch Pipeline jobs, not in regular Pipeline jobs.

⚠ **NOTE:** checkout scm only works correctly in Multibranch Pipeline or SCM-triggered Pipeline jobs.

⚠ **NOTE:** The input step holds an Agent executor while waiting — always wrap with timeout() to prevent blocking.

⚠ **NOTE:** The @Library annotation trailing underscore _ is required Groovy syntax — without it the file will not parse.

⚠ **NOTE:** vars/, src/, resources/ directory names in Shared Libraries are fixed — Jenkins looks for exactly these names.

⚠ **NOTE:** Never put credentials as plain text in Jenkinsfiles — always use withCredentials(). Git history is permanent.

⚠ **NOTE:** disableConcurrentBuilds() prevents two deploy stages running simultaneously on the same server.

⚠ **NOTE:** In Groovy: double-quoted strings expand ${variables}. Single-quoted strings do NOT.

⚠ **NOTE:** The Jenkins API token (for linter and curl) is generated from your user profile — not your login password.

⚠ **NOTE:** The -B flag in Maven commands (batch mode) suppresses download progress output in Console Output.

⚠ **NOTE:** Blue Ocean per-stage log is the biggest practical advantage — click the red stage, see only that stage's output.

⚠ **NOTE:** Replay is for debugging only — changes do not persist to the repository Jenkinsfile.

⚠ **NOTE:** Azure Repos webhook setup: Azure DevOps → Project Settings → Service Hooks → Jenkins subscription.

⚠ **NOTE:** Jenkins URL in Instance Configuration must be the public IP/hostname for Azure VM deployments, not localhost.

⚠ **NOTE: Credentials scope: Global = all jobs. System = Jenkins internal only. Folder = jobs in that folder only.**

⚠ **NOTE: When referencing a job inside a folder from another job, always use the full path: folder/subfolder/job-name.**

⚠ **NOTE: Pin Shared Library default version to a release tag (v1.2.0) in production — not main — to prevent bad commits breaking all pipelines.**